

Механизм объединения идентичных RPC-запросов в группы

Введение

Среди [RPC-запросов](#), обрабатываемых сервисом приложения, в заданный промежуток времени можно выделить такие, для которых возвращается идентичный ответ. С точки зрения оптимизации нагрузки на сервис, целесообразно выполнить только один RPC-запрос и вернуть ответ всем остальным. Для реализации этого функционала сервисный фреймворк предоставляет механизмы кэширования [методов бизнес-логики](#) и объединения RPC-запросов в группы.

[Кэширование](#) применяют, когда метод БЛ запрашивается часто, а потому его ответ можно поместить в промежуточное хранилище и возвращать на каждый идентичный запрос.

Напротив, механизм объединения RPC-запросов применяется, когда кэширование работает неэффективно из-за низкого процента попадания в кэш. Последнее происходит по двум причинам:

- низкое [время жизни кэша](#);
- частая [инвалидация кэша](#).

Далее в статье приведено описание механизма объединения RPC-запросов и примеры внедрения.

Как работает объединение RPC-запросов

Объединение RPC-запросов работает таким образом, что фактически сервис выполняет только один запрос из группы, а его ответ транслируется всем остальным.

При добавлении запроса в группу механизм автоматически генерирует на сервисе исключение [sbis::rpc::InternalTaskMergeRequestException \(res\)](#), где аргумент **res** — это номер уже выполняемого запроса группы. После генерации исключения сервис самостоятельно выполняет дальнейшую обработку запроса и доставку ему ответа.



Что такое номер RPC-запроса?

Любому запросу, который обрабатывает сервис, присваивается номер. Нумерация начинается с нуля, отсчет происходит от момента начала работы сервиса. Список запросов и их номера можно найти на панели администратора сервиса ["Управление облаком"](#) в разделе "Журнал логов".

На добавление запроса в группу уходит некоторое время. Если в группе получен ответ раньше, чем выполнено добавление, тогда такой RPC-запрос повторно ставится на выполнение сервисом.

Критерии отбора запросов, добавляемых в группу, зависят от прикладной задачи.

Как использовать механизм?

C++

1. Подключите сервисный модуль **RequestMerger** в состав вашего прикладного сервиса. Модуль включен в [SBIS SDK](#) и по умолчанию включен в шаблон `basic_platform` (см. [Состав SDK](#)). В составе модуля `RequestMerger` распространяется библиотека `sbis-request-merger`.
2. В файле, где в исходном коде C++ описана реализация метода БЛ, подключите заголовочный файл библиотеки:

```
1 #include <sbis-request-merger/request_merger.hpp>
```

3. Добавьте в начале тела метода БЛ конструкцию вида:

```
1 merger::RequestMerger req_merger( arg1, arg2 );
```

где

- `arg1` — произвольная строковая константа;
рекомендуется создавать такую константу, имя которой бы соответствовало функциональной области сервиса, в котором реализован метод БЛ;
- `arg2` — динамически сформированная строка.

По паре `arg1` и `arg2` механизм однозначно идентифицируется **ключ слияния** RPC-запросов в единую группу. Такой ключ не зависит от параметров вызова метода БЛ. Ответственность за формирование ключа находится только на прикладном разработчике.

Python

1. Импортируйте **RequestMerger-Py** в соответствующий сервисный модуль:

```
1 <bl_module id="cbfc8c32-dc7d-4935-9855-524934ed8a9d"
2     name="RequestMerger-Py"
3     url="$(SBISPlatformSDK)/bl-modules/RequestMerger-
  Py/RequestMerger-Py.s3mod"/>
```

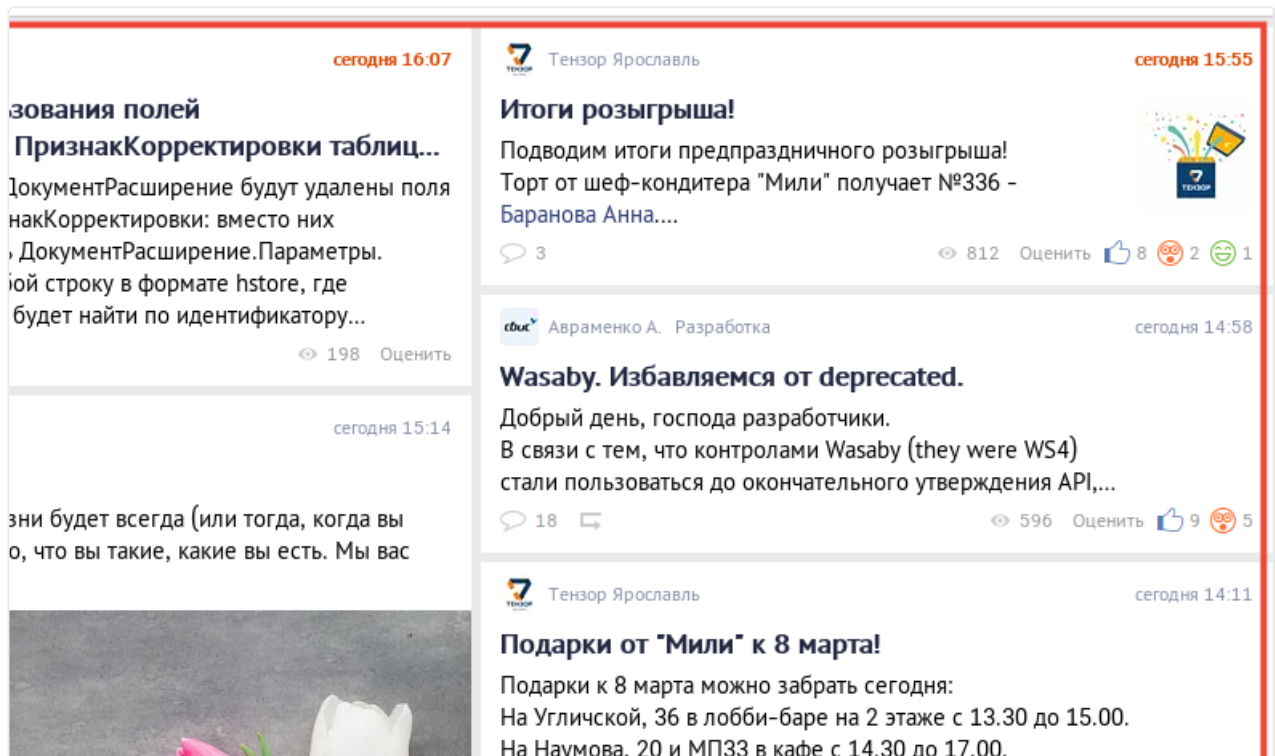
2. Используйте в коде:

```
1 import requestmerger
2     hash_params = 'blablabla'
3     with requestmerger.RequestMerger("Query", hash_params):
4         my_function()
```

Прикладной пример внедрения механизма для Сервиса представления

В ходе коммерческой эксплуатации было выявлено, что для [Сервиса представления](#) кэширование оказывается дорогостоящим решением по нескольким причинам:

- веб-страницы приложения СБИС формируются из динамически изменяемых данных. Например, главная страница online.sbis.ru может показывать непрочитанные сообщения, новостную ленту, новые задачи и другое (зависит от персональных настроек и прав доступа). Пользователю должны выводиться самые "свежие" данные, а поэтому хранить "устаревшие" не имеет смысла.



Панель отчета "Асинхронная очередь" отображает текущее состояние очередей асинхронных вызовов. Отчет выводит оперативную информацию, и пользователю так же нет смысла показывать устаревший отчет.

```
-bl1.unix.tensor.ru:20379
```

```
-bl2.unix.tensor.ru:20379
```

- для веб-страниц сложно определить какие результаты методов БЛ требуется кэшировать.

В рамках сервиса эффективнее применить механизм объединения RPC-запросов. С помощью механизма можно исключить повторное выполнение методов БЛ в заданный промежуток времени. Например, пользователь зажимает клавишу F5 для обновления страницы. В этом случае не имеет смысла обрабатывать запросы на каждое обновление, а выполнить их один раз. Другой пример, отчет "Асинхронная очередь" запрашивают несколько пользователей практически одновременно, тогда им можно отдать идентичный результат.

Далее приведены примеры как реализовано объединение запросов с одинаковыми строкой запроса, сессией, локализацией и версией клиента.

В файле [process.cpp](#) создана функция, которая вычисляет хэш для запросов веб-страниц:

```
1 RpcFile Service::Process( String const& request_uri ) const
2 {
3     wchar_t const* session_id = Session::Context().Get( icsSESSION_ID
4 );
5     wchar_t const* lang = Session::Context().Get( icsLANGUAGE );
6     merger::RequestMerger req_merger( PS_REQUEST_MERGER_SCOPE,
7                                     Format( L"Process|%1|%2|%3%"_sv,
8 request_uri,
9 session_id ? StringView( session_id ) : L""_sv,
10 lang ? StringView( lang ) : L""_sv ));
11     ...
```

В файле [process_package.cpp](#) описана функция, которая вычисляет хэш для запросов [rt-пакетов](#).

```
1 void Service::ProcessPackage( IDataConsumer& result, IContext const&
2 context, String const& request_uri ) const
3 {
4     StringView accept_encoding{ Session::Get( icsACCEPT_ENCODING ) };
5     bool need_gzip = ( !accept_encoding.empty() && accept_encoding.find(
6 L"gzip"_sv ) != StringView::npos );
7     merger::RequestMerger req_merger( PS_REQUEST_MERGER_SCOPE,
8                                     Format(
9 L"ProcessPackage|%1|%2%"_sv, request_uri, need_gzip ) );
10     ...
```